Architecture 1001: x86-64 Assembly SingleLocalVariable.c with GCC

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

SingleLocalVariable.c

Adding a single local variable

```c
int func(){
    int i = 0x5ca1ab1e;
    return i;
}
int main(){
    return func();
}
```

```
func:
   0x000055555555129 <+0>:     endbr64
   0x00005555555512d <+4>:     push   %rbp
   0x00005555555512e <+5>:     mov    %rsp,%rbp
   0x000055555555131 <+8>:     movl
    $0x5ca1ab1e,-0x4(%rbp)
   0x000055555555138 <+15>:    mov    -0x4(%rbp),%eax
   0x00005555555513b <+18>:    pop    %rbp
   0x00005555555513c <+19>:    retq
main:
   0x00005555555513d <+0>:     endbr64
   0x000055555555141 <+4>:     push   %rbp
   0x000055555555142 <+5>:     mov    %rsp,%rbp
   0x000055555555145 <+8>:     mov    $0x0,%eax
   0x00005555555514a <+13>:    callq  0x555555555129 <func>
   0x00005555555514f <+18>:    pop    %rbp
   0x000055555555150 <+19>:    retq
```

SingleLocalVariable.c Stack

- If we step *through* line 0x5555`55555131 in func(), the stack looks like this:

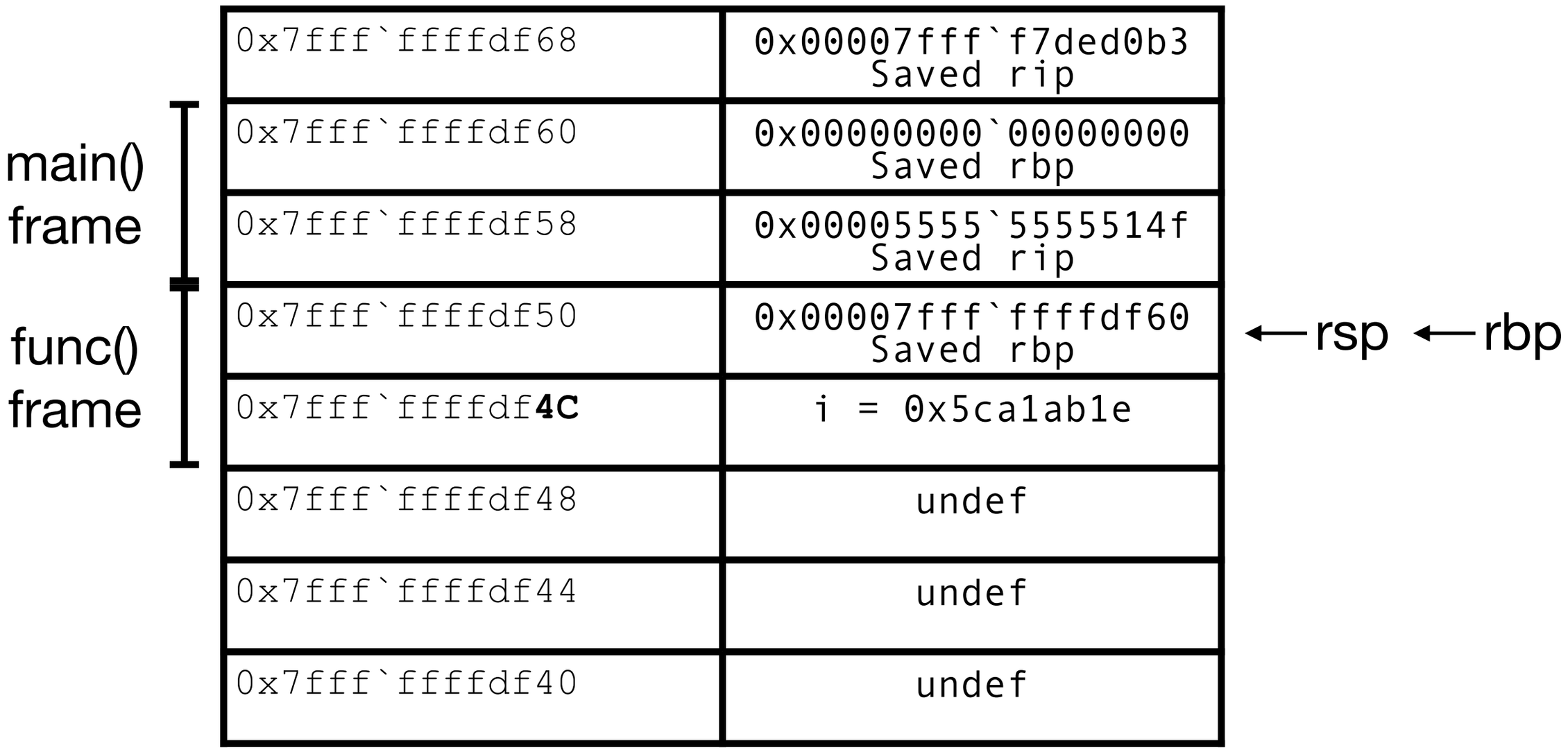


Figure 3.3: Stack Frame with Base Pointer

Position	Contents	Frame
`8n+16(%rbp)`	memory argument eightbyte n	Previous
	...	
`16(%rbp)`	memory argument eightbyte 0	
`8(%rbp)`	return address	Current
`0(%rbp)`	previous `%rbp` value	
`-8(%rbp)`	unspecified	
	...	
`0(%rsp)`	variable size	
`-128(%rsp)`	red zone	

"The 128-byte area beyond the location pointed to by %rsp is considered to be reserved and shall not be modified by signal or interrupt handlers. Therefore, functions may use this area for temporary data that is not needed across function calls. **In particular, leaf functions may use this area for their entire stack frame, rather than adjusting the stack pointer in the prologue and epilogue.** This area is known as the red zone."

From System V ABI: https://raw.githubusercontent.com/wiki/hjl-tools/x86-psABI/x86-64-psABI-1.0.pdf